C# / .NET Learning Roadmap

Topic Notes — Section 7: Asynchronous Programming

Topics: 7.1 through 7.9

Date: 29 June 2026

Section 7 — Asynchronous Programming

This section covers all nine roadmap topics: 7.1 (Threads vs Tasks), 7.2 (Task and Task<T>), 7.3 (async/await), 7.4 (Task.WhenAll, Task.WhenAny), 7.5 (CancellationToken), 7.6 (Exception handling in async), 7.7 (IAsyncEnumerable<T>), 7.8 (Common async pitfalls), 7.9 (Task.Run vs async).

Teaching order: Topics taught in dependency order. 7.1 provides the conceptual foundation. 7.2 and 7.3 are the core mechanics. All other topics build on those.

7.1. Threads vs Tasks (Conceptual)

Builds on: 1.4 (value vs reference types — Task is a reference type), 1.8 (loops — async is about doing work without blocking). This is a conceptual foundation topic.

The problem async solves

Without async, a server assigns a thread to each request and that thread waits idle while the operation completes. Threads are expensive — each consumes ~1MB of stack memory and has scheduling overhead. Under heavy load, the thread pool exhausts and new requests queue up or are rejected.

```
Request 1 -> Thread 1 -> [waiting for DB... 200ms idle] -> returns response
Request 2 -> Thread 2 -> [waiting for API... 500ms idle] -> returns response
Request 3 -> Thread 3 -> [waiting for file... 100ms idle] -> returns response
```

Threads — the low-level mechanism

A thread is an OS-level construct — a sequence of instructions the CPU can execute. The .NET thread pool maintains reusable threads to avoid creation/destruction overhead.

```
// Creating a thread manually -- rarely done in modern .NET:
var thread = new Thread(() =>
{
    Console.WriteLine($"Running on thread {Thread.CurrentThread.ManagedThreadId}");
    Thread.Sleep(1000);   // blocks this thread for 1 second
});
thread.Start();
thread.Join();   // waits for thread to finish
```

Problems with raw threads for I/O: threads waiting for I/O are blocked — they consume memory but do no useful work. Thread creation is expensive (~1ms, ~1MB stack). Context switching has CPU overhead.

Tasks — the abstraction over threads

A Task represents a unit of work that will complete in the future. It is a promise of a result, not a thread. The key insight: I/O-bound tasks do NOT need a thread while waiting. The OS handles the I/O at the hardware level and notifies .NET when done. The thread is returned to the pool during the wait.

```
// With async, the same thread pool handles hundreds of concurrent requests:
Request 1 -> Thread 1 -> [starts DB query] -> Thread 1 released back to pool
```

```
                                                  -> [DB query runs at hardware level]
                                                  -> Thread 4 (from pool) -> handles response
Request 2 -> Thread 1 (reused!) -> [starts API call] -> Thread 1 released again
```

CPU-bound vs I/O-bound work

```
// I/O-bound -- use async/await directly:
public async Task<string> GetDataAsync()
{
    var response = await _httpClient.GetStringAsync("https://api.example.com/data");
    return response;   // no thread consumed during the HTTP call
}

// CPU-bound -- use Task.Run to move to background thread:
public async Task<int[]> SortLargeArrayAsync(int[] data)
{
    return await Task.Run(() =>
    {
        Array.Sort(data);   // CPU-intensive -- runs on thread pool thread
        return data;
    });
}
```

Key Takeaways for 7.1

- Threads are OS-level execution units — expensive, blocking during I/O
- Tasks are promises of future work — lightweight, don't block during I/O
- I/O-bound work: async/await -- thread is freed during the wait
- CPU-bound work: Task.Run -- offloaded to a thread pool thread
- async/await lets a server handle hundreds of concurrent requests with a small thread pool

```
                  | Thread                     | Task
------------------+----------------------------+---------------------------
What it is        | OS execution unit          | Promise of future result
Creation cost     | High (~1ms, ~1MB stack)    | Very low
I/O waiting       | Blocks the thread          | Releases the thread
Best for          | Long-running background    | Short async operations
Modern usage      | Rare                       | Standard everywhere
```

Memory aid: A thread is a waiter who stands at your table doing nothing while the kitchen cooks your food. A Task is a buzzer — the waiter gives you a buzzer and goes to serve other tables. When your food is ready, the buzzer fires and any available waiter brings it.

7.2. Task and Task<T>

Builds on: 7.1 (threads vs tasks), 6.1 (delegates — Task.Run takes a delegate), 3.1 (generics — Task<T> is generic).

What is Task?

Task represents an async operation producing no value (like void). Task<T> represents an async operation producing a value of type T.

```
Task SaveAsync() { ... }            // async void equivalent
Task<Order> GetOrderAsync(int id) { ... }
Task<List<Order>> GetAllOrdersAsync() { ... }
Task<bool> ExistsAsync(int id) { ... }
```

Task states

```
// A Task moves through states:
// Created -> Running -> [RanToCompletion | Faulted | Cancelled]

var task = Task.Run(() => Thread.Sleep(1000));
Console.WriteLine(task.Status);     // Running
task.Wait();                         // blocks current thread until done
Console.WriteLine(task.Status);     // RanToCompletion

task.IsCompleted              // true if any terminal state
task.IsCompletedSuccessfully  // true only if RanToCompletion
task.IsFaulted                // true if threw an exception
task.IsCanceled               // true if cancelled
```

Creating Tasks

```
// Task.Run -- runs a delegate on the thread pool:
Task task = Task.Run(() => Console.WriteLine("hello"));
Task<int> computation = Task.Run(() => 42);

// Task.FromResult -- already-completed Task<T>:
Task<Order> completed = Task.FromResult(new Order { Id = 1 });
// Useful when implementing interfaces that return Task synchronously

// Task.CompletedTask -- pre-completed Task (no allocation):
Task done = Task.CompletedTask;

// Task.Delay -- non-blocking wait:
await Task.Delay(1000);   // releases thread during the 1 second wait
// vs Thread.Sleep(1000) which BLOCKS the thread
```

Task.FromResult and Task.CompletedTask — synchronous implementations

```
// Synchronous in-memory implementation for tests:
public class InMemoryOrderRepository : IOrderRepository
{
    private readonly Dictionary<int, Order> _store = new();

    public Task<Order?> GetByIdAsync(int id)
    {
        _store.TryGetValue(id, out var order);
        return Task.FromResult(order);   // wraps sync result in a Task
    }
}
```

Key Takeaways for 7.2

- Task = async void equivalent -- an operation with no return value
- Task<T> = async operation that produces a value of type T
- Task.Run() -- runs work on the thread pool (CPU-bound use case)
- Task.FromResult() -- wraps a synchronous result in a completed Task
- Task.CompletedTask -- pre-completed Task with no allocation
- Task.Delay() -- non-blocking wait (vs Thread.Sleep which blocks)
- Never use .Wait() or .Result inside async methods -- causes deadlocks

Memory aid: Task is a numbered ticket at a deli counter. You get the ticket (Task), go sit down, and wait for your number to be called (await). You don't stand at the counter blocking everyone else.

Debugging Tips

Symptom	Cause	Fix
Task.Result or .Wait() causes deadlock	Blocking on async code in a SynchronizationContext environment	Use await instead of .Result or .Wait() throughout the call chain
Task.Run returns before work finishes	Forgot to await the Task returned by Task.Run	Always await Task.Run -- store the Task and await it
Task.FromResult with null causes NullReferenceException	Passing null to Task.FromResult where T is non-nullable	Use Task.FromResult<T?>(null) with nullable type parameter

7.3. async/await Keywords

Builds on: 7.2 (Task -- async/await is syntactic sugar over Task), 6.4 (lambdas -- async lambdas follow the same pattern), 1.13 (exception handling -- async exceptions work differently).

What async/await does

async and await are compiler keywords that transform your sequential-looking code into a state machine that handles Task continuations automatically. You write sequential code; the compiler does the complex callback wiring.

```
// What you write:
public async Task<Order> GetOrderAsync(int id)
{
    var order = await _context.Orders.FindAsync(id);
    if (order is null) throw new NotFoundException("Order", id);
    return order;
}

// What happens at the await point:
// 1. Starts _context.Orders.FindAsync(id)
// 2. Returns a Task<Order> to the caller IMMEDIATELY
// 3. Thread is released back to pool
// 4. When FindAsync completes, method resumes
// 5. Completes the returned Task<Order> with the result
```

async method rules

```
// async method must return Task, Task<T>, ValueTask, or void (event handlers only):
public async Task DoSomethingAsync() { ... }
public async Task<Order> GetOrderAsync(int id) { ... }
public async ValueTask<int> GetCountAsync() { ... }
public async void OnButtonClick(object sender, EventArgs e) { ... }  // event only

// async without await -- compiler warns, remove async keyword:
public async Task<int> ShouldNotBeAsync() { return 42; }  // warning

// Correct when no await needed:
public Task<int> ReturnsImmediately() => Task.FromResult(42);
```

The async all the way pattern

```
// The entire call chain must be async:
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
{
    var order = await _orderService.GetByIdAsync(id);  // async
    return order is null ? NotFound() : Ok(order);
}

public async Task<Order?> GetByIdAsync(int id)
    => await _repo.GetByIdAsync(id);  // async

public async Task<Order?> GetByIdAsync(int id)
    => await _context.Orders.FindAsync(id);  // async
```

async in ASP.NET Core controllers

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> GetAll([FromQuery] int page = 1)
    {
        var orders = await _service.GetPagedAsync(page, pageSize: 20);
        return Ok(orders);
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> GetById(int id)
    {
        var order = await _service.GetByIdAsync(id);
        return order is null ? NotFound() : Ok(order);
    }

    [HttpPost]
    public async Task<IActionResult> Create(CreateOrderDto dto)
    {
        var order = await _service.CreateAsync(dto);
```

```
        return CreatedAtAction(nameof(GetById), new { id = order.Id }, order);
    }
}
```

Key Takeaways for 7.3

- async marks a method as asynchronous -- the compiler generates a state machine
- await suspends the method, releases the thread, and resumes when the Task completes
- Async methods return Task (void equivalent) or Task<T> (value equivalent)
- async void ONLY for event handlers -- exceptions in async void are uncatchable
- Async all the way -- the entire call chain must be async

Memory aid: async/await is like placing an online order. You click Order (call async method), get a confirmation (Task returned), go do other things (thread freed), and get a notification when it ships (method resumes).

Debugging Tips

Symptom	Cause	Fix
Method marked async but compiler warns lacks await	No await inside -- method runs synchronously	Remove async and return Task.FromResult(), or add the missing await
async void method exceptions crash the app	async void exceptions cannot be caught by callers	Change to async Task and propagate the exception properly
Response returned before async work finishes	Forgot await on the inner async call	Trace the call chain and ensure every async call is awaited

7.4. Task Composition (Task.WhenAll, Task.WhenAny)

Builds on: 7.2 (Task -- composition works on Tasks), 7.3 (async/await -- you await the composed results), 7.6 (exception handling -- WhenAll aggregates exceptions).

What is Task composition and why does it exist?

Sequential awaits run one operation at a time. When operations are independent, this wastes time. Task.WhenAll and Task.WhenAny run Tasks concurrently.

```
// Sequential -- total time = 200ms + 300ms + 150ms = 650ms:
var orders    = await _orderRepo.GetByUserAsync(userId);      // 200ms
var profile   = await _userRepo.GetProfileAsync(userId);      // 300ms
var analytics = await _analyticsService.GetStatsAsync(userId); // 150ms

// Concurrent with WhenAll -- total time = max(200, 300, 150) = 300ms:
var ordersTask    = _orderRepo.GetByUserAsync(userId);
var profileTask   = _userRepo.GetProfileAsync(userId);
var analyticsTask = _analyticsService.GetStatsAsync(userId);

await Task.WhenAll(ordersTask, profileTask, analyticsTask);

// .Result is safe here -- tasks are guaranteed complete:
```

```
var orders    = ordersTask.Result;
var profile   = profileTask.Result;
var analytics = analyticsTask.Result;
```

Task.WhenAll with same-type collections

```
// WhenAll with same-type Tasks returns T[]:
Task<decimal>[] revenueTasks = regions
    .Select(r => _service.GetRevenueAsync(r))
    .ToArray();

decimal[] revenues = await Task.WhenAll(revenueTasks);
decimal total = revenues.Sum();

// Process all orders concurrently:
var processTasks = orders.Select(o => _processor.ProcessAsync(o));
await Task.WhenAll(processTasks);

// Caution with large collections -- batch to avoid overwhelming services:
foreach (var batch in orderIds.Chunk(10))
{
    var tasks = batch.Select(id => _processor.ProcessAsync(id));
    await Task.WhenAll(tasks);   // 10 at a time
}
```

Task.WhenAny — complete when first Task finishes

```
// Timeout pattern:
var operationTask = _slowService.GetDataAsync();
var timeoutTask   = Task.Delay(TimeSpan.FromSeconds(5));

Task completedFirst = await Task.WhenAny(operationTask, timeoutTask);

if (completedFirst == timeoutTask)
    throw new TimeoutException("Operation timed out after 5 seconds");

var result = await operationTask;   // safe -- already completed

// Progress reporting -- handle each as it completes:
var tasks = orders.Select(o => _processor.ProcessAsync(o)).ToList();

while (tasks.Count > 0)
{
    Task<ProcessResult> completed = await Task.WhenAny(tasks);
    tasks.Remove(completed);
    var result = await completed;
    Console.WriteLine($"Processed: {result.OrderId}");
}
```

Dashboard data load scenario

```
public async Task<DashboardViewModel> GetDashboardAsync(int userId)
{
    var ordersTask  = _orderRepo.GetRecentAsync(userId, days: 30);
    var revenueTask = _orderRepo.GetRevenueAsync(userId);
```

```
    var profileTask = _userRepo.GetProfileAsync(userId);
    var alertsTask  = _alertService.GetActiveAlertsAsync(userId);

    await Task.WhenAll(ordersTask, revenueTask, profileTask, alertsTask);

    return new DashboardViewModel
    {
        RecentOrders  = ordersTask.Result,
        TotalRevenue  = revenueTask.Result,
        Profile       = profileTask.Result,
        ActiveAlerts  = alertsTask.Result,
        LoadedAt      = DateTime.UtcNow
    };
}
```

Key Takeaways for 7.4

- Task.WhenAll -- runs Tasks concurrently, completes when ALL finish
- Task.WhenAny -- completes when ANY ONE Task finishes first
- Start Tasks without await, then WhenAll -- this is what enables concurrency
- .Result is safe after WhenAll because the Task is guaranteed complete
- For large collections, batch with Chunk to avoid overwhelming downstream services
- WhenAny is the standard timeout pattern in .NET

```
               | Task.WhenAll             | Task.WhenAny
---------------+--------------------------+-----------------------------
Completes when | All Tasks done           | First Task done
Result         | Task[] or T[]            | The first completed Task
Use for        | Independent parallel     | Timeout, race, progress
Error behaviour| Aggregates exceptions    | Only first Task's exception
```

Memory aid: WhenAll is like waiting for all your friends to arrive before starting dinner. WhenAny is like a race -- you start eating as soon as the first friend arrives.

Debugging Tips

Symptom	Cause	Fix
WhenAll runs sequentially instead of concurrently	Tasks were awaited inside Select before being passed to WhenAll	Store tasks first without awaiting, then pass to WhenAll
WhenAll throws only one exception when multiple tasks failed	await unwraps only the first from AggregateException	Check each task with .Where(t => t.IsFaulted).Select(t => t.Exception!.InnerException)
WhenAny returns a faulted task unexpectedly	The first task to complete threw an exception	Await the returned task inside try/catch to handle the exception
Overwhelming database with concurrent requests	Running WhenAll on too many tasks simultaneously	Batch with Chunk(n) and WhenAll per batch

7.5. CancellationToken

Builds on: 7.3 (async/await -- cancellation is passed through async call chains), 7.2 (Task -- CancellationToken causes Tasks to transition to Cancelled state), 2.9 (interfaces -- CancellationToken is passed through method signatures by convention).

What is CancellationToken and why does it exist?

CancellationToken is .NET's cooperative cancellation mechanism. The code that triggers cancellation signals it; the code doing the work checks for it and stops cleanly. Without cancellation, async operations run to completion regardless of whether the result is still needed.

```
// CancellationTokenSource -- the controller:
var cts = new CancellationTokenSource();
CancellationToken token = cts.Token;

var task = _service.GetDataAsync(token);

cts.Cancel();   // trigger cancellation from outside

try
{
    var result = await task;
}
catch (OperationCanceledException)
{
    Console.WriteLine("Operation was cancelled");
}
finally
{
    cts.Dispose();   // always dispose CancellationTokenSource
}

// Timeout cancellation:
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
// or:
using var cts = new CancellationTokenSource();
cts.CancelAfter(TimeSpan.FromSeconds(5));
```

ASP.NET Core -- HttpContext.RequestAborted

```
// CancellationToken automatically bound from HttpContext.RequestAborted:
[HttpGet]
public async Task<IActionResult> GetOrders(CancellationToken cancellationToken)
{
    // Cancelled when: client disconnects, request times out, server shutting down
    var orders = await _service.GetOrdersAsync(cancellationToken);
    return Ok(orders);
}
```

Passing CancellationToken through the call chain

```
// Controller -> Service -> Repository -> EF Core:
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id, CancellationToken ct)
    => Ok(await _orderService.GetByIdAsync(id, ct));
```

```
public async Task<Order?> GetByIdAsync(int id, CancellationToken ct)
{
    var order = await _repo.GetByIdAsync(id, ct);
    await _auditService.RecordAccessAsync(id, ct);
    return order;
}

public async Task<Order?> GetByIdAsync(int id, CancellationToken ct)
    => await _context.Orders.FirstOrDefaultAsync(o => o.Id == id, ct);
```

Checking cancellation manually

```
public async Task ProcessOrdersAsync(IEnumerable<Order> orders, CancellationToken ct)
{
    foreach (var order in orders)
    {
        // ThrowIfCancellationRequested -- throws OperationCanceledException:
        ct.ThrowIfCancellationRequested();
        await ProcessSingleOrderAsync(order, ct);
    }
}

// IsCancellationRequested -- when you want graceful partial results:
foreach (var order in orders)
{
    if (ct.IsCancellationRequested) break;
    results.Add(await ProcessSingleOrderAsync(order, ct));
}
```

Linking CancellationTokens

```
// Combine user's token with a per-operation timeout:
public async Task<Data> GetWithTimeoutAsync(int id, CancellationToken userCt)
{
    using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(10));
    using var linkedCts  = CancellationTokenSource.CreateLinkedTokenSource(
        userCt, timeoutCts.Token);

    // linkedCts.Token is cancelled when EITHER source cancels:
    return await _service.GetDataAsync(id, linkedCts.Token);
}
```

CancellationToken.None and BackgroundService

```
// CancellationToken.None -- never cancelled:
await _service.GetDataAsync(CancellationToken.None);

// Default parameter -- same effect, cleaner API:
public async Task<Data> GetDataAsync(CancellationToken ct = default) { ... }

// BackgroundService -- stoppingToken cancelled on app shutdown:
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
```

```
        try
        {
            var orders = await _repo.GetPendingAsync(stoppingToken);
            foreach (var order in orders)
            {
                stoppingToken.ThrowIfCancellationRequested();
                await _processor.ProcessAsync(order, stoppingToken);
            }
            await Task.Delay(TimeSpan.FromSeconds(30), stoppingToken);
        }
        catch (OperationCanceledException) { break; }
    }
}
```

Key Takeaways for 7.5

- CancellationTokenSource triggers cancellation; CancellationToken is the signal passed to operations
- In ASP.NET Core, inject CancellationToken into action methods -- automatically bound from HttpContext.RequestAborted
- Pass CancellationToken through every layer: controller -> service -> repository -> EF Core
- ct.ThrowIfCancellationRequested() in loops; ct.IsCancellationRequested for graceful partial results
- CancellationTokenSource.CreateLinkedTokenSource combines multiple cancellation sources
- Always Dispose CancellationTokenSource -- use using

```
Member                          | Purpose
--------------------------------+-----------------------------------------
new CancellationTokenSource()   | Creates cancellation controller
cts.Cancel()                    | Triggers cancellation immediately
cts.CancelAfter(delay)          | Triggers cancellation after delay
cts.Token                       | Gets the token to pass to operations
ct.IsCancellationRequested      | Check without throwing
ct.ThrowIfCancellationReqstd()| Check and throw if cancelled
ct.Register(callback)           | Run code when cancelled
CancellationToken.None          | Never-cancellable token
```

Memory aid: CancellationToken is like a fire alarm pull station. The source is the pull station -- only authorised code can pull it. The token is the alarm signal that everyone in the building listens to and responds to by stopping cleanly.

Debugging Tips

Symptom	Cause	Fix
Operation continues after client disconnects	CancellationToken not passed to EF Core or HttpClient calls	Pass ct to every async method in the chain
OperationCanceledException not caught	Catching TaskCanceledException only, missing base class	Catch OperationCanceledException -- it is the base of TaskCanceledException
Application hangs on shutdown	BackgroundService not checking stoppingToken	Pass stoppingToken to all inner async calls and check in loops

Symptom	Cause	Fix
Linked token does not cancel when source cancels	Source was disposed before the linked token was used	Keep CancellationTokenSource alive for the duration of the operation

7.6. Exception Handling in Async Code

Builds on: 7.3 (async/await -- exceptions surface when you await), 7.4 (Task.WhenAll -- aggregates multiple exceptions), 7.5 (CancellationToken -- OperationCanceledException is a special case), 1.13 (exception handling fundamentals).

How exceptions work differently in async code

In synchronous code, an exception thrown inside a method immediately propagates up the call stack. In async code, the exception is captured inside the Task and only surfaces when you await it.

```
// Async -- exception captured in the Task:
public async Task<string> GetDataAsync()
{
    throw new InvalidOperationException("Failed");   // captured in the Task
}

Task<string> task = GetDataAsync();   // no exception yet -- returns a Task

// Exception only surfaces when awaited:
try { var result = await task; }
catch (InvalidOperationException) { /* caught here */ }
```

try/catch/finally with await

```
public async Task<Order?> GetOrderSafelyAsync(int id)
{
    try
    {
        return await _repo.GetByIdAsync(id);
    }
    catch (SqlException ex)
    {
        _logger.LogError(ex, "Database error fetching order {Id}", id);
        throw;   // re-throw -- preserve stack trace
    }
    catch (TimeoutException ex)
    {
        _logger.LogWarning(ex, "Timeout fetching order {Id}", id);
        return null;   // swallow intentionally
    }
}

// await using for IAsyncDisposable:
public async Task ProcessWithCleanupAsync(int orderId)
{
    await using var connection = await _pool.GetConnectionAsync();
```

```
    await _processor.ProcessAsync(orderId, connection);
    // connection.DisposeAsync() called automatically
}
```

async void -- the dangerous exception case

```
// DANGEROUS -- exception cannot be caught by caller:
public async void FireAndForget()
{
    await Task.Delay(100);
    throw new Exception("This will crash the process!");
}

// CORRECT -- return Task:
public async Task FireAndForgetAsync()
{
    await Task.Delay(100);
    throw new Exception("This CAN be caught");
}

// ONLY legitimate async void -- event handler with full try/catch:
private async void OnOrderPlaced(object? sender, OrderPlacedEventArgs e)
{
    try
    {
        await _emailService.SendConfirmationAsync(e.CustomerEmail);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Failed to send confirmation");
        // Don't re-throw -- would crash the process
    }
}
```

Task.WhenAll -- AggregateException and multiple failures

```
// await WhenAll only surfaces the FIRST exception:
try
{
    await Task.WhenAll(tasks);
}
catch
{
    // Inspect ALL failed tasks:
    var failures = tasks
        .Where(t => t.IsFaulted)
        .Select(t => t.Exception!.InnerException!)
        .ToList();

    foreach (var ex in failures)
        _logger.LogError(ex, "Processing failed");

    throw new AggregateException("Multiple failures", failures);
}
```

Exception filters and retry pattern

```
// when filters -- conditional exception handling:
catch (SqlException ex) when (ex.Number == 1205)   // only deadlocks
{
    await Task.Delay(100, ct);
    return await GetOrderAsync(id, ct);   // retry once
}
catch (OperationCanceledException) when (ct.IsCancellationRequested)
{
    return null;   // our cancellation
}

// Retry with exponential backoff:
public static async Task<T> RetryAsync<T>(
    Func<Task<T>> operation, int maxAttempts = 3,
    TimeSpan? delay = null, CancellationToken ct = default)
{
    var retryDelay = delay ?? TimeSpan.FromSeconds(1);
    Exception? lastException = null;

    for (int attempt = 1; attempt <= maxAttempts; attempt++)
    {
        try { return await operation(); }
        catch (OperationCanceledException) { throw; }  // never retry cancellation
        catch (Exception ex) when (attempt < maxAttempts)
        {
            lastException = ex;
            await Task.Delay(retryDelay, ct);
            retryDelay *= 2;   // exponential backoff
        }
    }
    throw new Exception($"All {maxAttempts} attempts failed", lastException);
}
```

Key Takeaways for 7.6

- Async exceptions are captured in the Task and only surface when awaited
- try/catch/finally works normally around await expressions
- async void exceptions cannot be caught -- always wrap body in try/catch and never re-throw
- Task.WhenAll with multiple failures -- await only surfaces the first; inspect all tasks for all failures
- when filters allow conditional exception handling without catching and re-throwing
- OperationCanceledException should almost never be swallowed -- let it propagate

```
Scenario                    | Exception behaviour
----------------------------+-------------------------------------------
await failedTask            | Unwraps and throws first exception
await Task.WhenAll(tasks)   | Throws first, others in AggregateException
async void throws           | Thrown on SynchronizationContext -- crashes
async Task throws           | Captured in Task, surfaced on await
OperationCanceledException  | Special -- signals cancellation, propagate it
```

Memory aid: In async code, exceptions travel in sealed envelopes (Tasks). You cannot open the envelope until you await it. WhenAll collects multiple envelopes but only shows you the first letter -- you have to open all envelopes yourself to read everything that went wrong.

Debugging Tips

Symptom	Cause	Fix
Exception from async void crashes the process	Unhandled exception in async void escapes the context	Wrap entire async void body in try/catch; change to async Task if possible
AggregateException with multiple inner exceptions after WhenAll	Multiple tasks failed -- await only surfaces the first	Check all tasks with .Where(t => t.IsFaulted).Select(t => t.Exception!.InnerException)
throw ex loses the original stack trace	Re-throwing with the exception variable resets the stack	Use throw; alone to preserve the original stack trace
OperationCanceledException caught unintentionally	Broad catch(Exception) catches cancellation too	Add separate catch(OperationCanceledException) before general catch and re-throw

7.7. IAsyncEnumerable<T> and Async Streams

Builds on: 7.3 (async/await -- async streams extend await into iteration), 4.5 (IEnumerable<T> -- IAsyncEnumerable<T> is the async counterpart), 7.5 (CancellationToken -- async streams accept cancellation), 1.8 (loops -- await foreach replaces foreach for async streams).

What is IAsyncEnumerable<T> and why does it exist?

IEnumerable<T> produces items synchronously. IAsyncEnumerable<T> produces items asynchronously -- each item may require an I/O operation before it's available, and items are consumed as they arrive without loading everything into memory.

```
// Without IAsyncEnumerable -- load everything into memory first:
List<Order> orders = await _context.Orders.ToListAsync();  // all 1M rows loaded
foreach (var order in orders)
    await ProcessAsync(order);

// With IAsyncEnumerable -- stream and process one at a time:
await foreach (var order in GetOrdersAsync(ct))
    await ProcessAsync(order);  // total memory: one row at a time
```

async iterator -- producing items with yield return

```
public async IAsyncEnumerable<Order> GetPendingOrdersAsync(
    [EnumeratorCancellation] CancellationToken ct = default)
{
    int page = 1;
    const int pageSize = 100;

    while (true)
    {
```

```
        var batch = await _context.Orders
            .Where(o => o.Status == "Pending")
            .OrderBy(o => o.Id)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .ToListAsync(ct);

        foreach (var order in batch)
            yield return order;   // yield each item to the consumer

        if (batch.Count < pageSize) break;   // last page reached
        page++;
    }
}
```

await foreach -- consuming async streams

```
// Basic consumption:
await foreach (var item in asyncSequence)
    await ProcessAsync(item);

// With CancellationToken:
await foreach (var item in asyncSequence.WithCancellation(ct))
    await ProcessAsync(item, ct);

// EF Core AsAsyncEnumerable -- streams rows from DB without bulk load:
await foreach (var order in _context.Orders
    .Where(o => o.Status == "Pending")
    .AsAsyncEnumerable()
    .WithCancellation(ct))
{
    await ProcessAsync(order, ct);
}
```

Real use cases

```
// Streaming export endpoint -- ASP.NET Core streams response progressively:
[HttpGet("export/csv")]
public async Task ExportCsvAsync([FromQuery] ExportFilter filter, CancellationToken ct)
{
    Response.ContentType = "text/csv";
    Response.Headers.Add("Content-Disposition", "attachment; filename=orders.csv");

    await using var writer = new StreamWriter(Response.Body);
    await writer.WriteLineAsync("Id,Date,Customer,Amount,Status");

    await foreach (var row in _exportService.StreamOrdersAsync(filter, ct))
        await writer.WriteLineAsync(
            $"{row.Id},{row.Date},{row.Customer},{row.Amount},{row.Status}");
    // Client starts receiving CSV data immediately
}

// Polling / live data feed:
public async IAsyncEnumerable<SensorReading> ReadSensorAsync(
    TimeSpan interval,
```

```
    [EnumeratorCancellation] CancellationToken ct = default)
{
    while (!ct.IsCancellationRequested)
    {
        yield return await _sensor.ReadAsync(ct);
        await Task.Delay(interval, ct);
    }
}
```

Key Takeaways for 7.7

- IAsyncEnumerable<T> = async stream -- items produced and consumed one at a time with async gaps
- await foreach = the consumption syntax for IAsyncEnumerable<T>
- async IAsyncEnumerable<T> + yield return = async iterator -- produces items lazily
- [EnumeratorCancellation] attribute wires cancellation between WithCancellation(ct) and the iterator
- EF Core's AsAsyncEnumerable() streams rows from the database without loading all into memory
- Use when datasets are large, streaming is needed, or Time To First Item matters

```
               | IEnumerable&lt;T&gt;     | IAsyncEnumerable&lt;T&gt;
---------------+--------------------+---------------------
Items produced | Synchronously      | Asynchronously
Consume with   | foreach            | await foreach
Create with    | yield return       | async + yield return
Cancellation   | Not built-in       | WithCancellation(ct)
EF Core        | .ToList()          | .AsAsyncEnumerable()
Memory         | All at once        | One at a time
```

Memory aid: IEnumerable is a fruit bowl -- all the fruit is there at once. IAsyncEnumerable is a fruit delivery service -- they bring one piece at a time from the warehouse. You never need a bowl big enough for all of them.

Debugging Tips

Symptom	Cause	Fix
await foreach gives compile error	Type does not implement IAsyncEnumerable<T>	Ensure the method returns IAsyncEnumerable<T> not Task<IEnumerable<T>>
Cancellation via WithCancellation has no effect	Missing [EnumeratorCancellation] attribute on the parameter	Add [EnumeratorCancellation] to the CancellationToken parameter in the iterator
Memory usage still high despite async streaming	Called ToListAsync() on the stream somewhere	Trace where ToListAsync() is called and replace with streaming consumption
EF Core AsAsyncEnumerable() throws outside DbContext lifetime	DbContext was disposed before the stream was fully consumed	Ensure DbContext lifetime covers the entire streaming operation

7.8. Common Async Pitfalls (ConfigureAwait, Sync-over-Async, Deadlocks)

Builds on: 7.3 (async/await -- pitfalls arise from misunderstanding how await works), 7.2 (Task -- .Wait() and .Result are the root of most deadlocks), 7.6 (exception handling -- async void covered there).

Pitfall 1 -- Sync-over-Async and Deadlocks

Calling .Wait() or .Result on a Task inside a SynchronizationContext environment causes a deadlock:

```
// DEADLOCKS in ASP.NET Classic, WinForms, WPF, xUnit test runner:
public string GetData()
{
    return GetDataAsync().Result;   // DEADLOCK
}

// Why it deadlocks:
// 1. GetData() calls GetDataAsync().Result
// 2. .Result blocks thread T1 (the original context thread)
// 3. GetDataAsync() hits await, captures the SynchronizationContext
// 4. Async operation completes -- tries to resume on T1
// 5. T1 is blocked by .Result -- waiting for GetDataAsync()
// 6. GetDataAsync() can't complete because T1 is blocked
// DEADLOCK

// FIX -- always await, never block:
public async Task<string> GetDataAsync()
    => await GetInnerDataAsync();
```

Pitfall 2 -- ConfigureAwait(false)

```
// Without ConfigureAwait(false) -- resumes on original context:
public async Task<string> LibraryMethodAsync()
{
    var data = await _httpClient.GetStringAsync(url);
    return data.ToUpperInvariant();
}

// With ConfigureAwait(false) -- resumes on thread pool (library code):
public async Task<string> LibraryMethodAsync()
{
    var data = await _httpClient.GetStringAsync(url).ConfigureAwait(false);
    return data.ToUpperInvariant();
}

// Rules:
// Library code (NuGet packages): always use ConfigureAwait(false)
// ASP.NET Core application code: not needed -- no SynchronizationContext
// WinForms/WPF: use ConfigureAwait(false) for non-UI work only
```

All 7 pitfalls -- summary

```
// PITFALL 1 -- SYNC-OVER-ASYNC:
var result = GetDataAsync().Result;   // deadlock risk
// FIX: var result = await GetDataAsync();

// PITFALL 2 -- CONFIGUREAWAIT (library code):
await SomeOp().ConfigureAwait(false);  // add in all library awaits

// PITFALL 3 -- ASYNC VOID:
public async void DoWork() { }        // never, except event handlers
// FIX: public async Task DoWorkAsync() { }

// PITFALL 4 -- FIRE AND FORGET without care:
_emailService.SendAsync(email);        // exception lost silently
// FIX: _ = SendSafeAsync(email);     // explicit discard + safe wrapper

// PITFALL 5 -- NOT AWAITING IN LOOPS:
foreach (var item in items) ProcessAsync(item);  // tasks discarded
// FIX: await Task.WhenAll(items.Select(i => ProcessAsync(i)));

// PITFALL 6 -- ASYNC IN CONSTRUCTOR:
public Service() { _data = await GetDataAsync(); }  // compile error
// FIX: public static async Task<Service> CreateAsync() { ... }

// PITFALL 7 -- Parallel.ForEach WITH ASYNC LAMBDA:
Parallel.ForEach(items, async item => await ProcessAsync(item));  // async void
// FIX: await Parallel.ForEachAsync(items, ct, async (item, t) => ...);
```

Key Takeaways for 7.8

- Sync-over-async (.Result, .Wait()) causes deadlocks in SynchronizationContext environments -- always await
- ConfigureAwait(false) required in library code, optional in ASP.NET Core application code
- async void ONLY for event handlers, always wrap body in try/catch
- Fire and forget -- always use _ = discard + safe wrapper method with try/catch
- Not awaiting in loops -- every async call must be awaited, sequentially or via WhenAll
- Constructors -- use async factory method, lazy initialisation, or IHostedService
- Parallel.ForEach does not work with async lambdas -- use Task.WhenAll or Parallel.ForEachAsync

Memory aid: Sync-over-async is like a drawbridge operator who needs a boat to pass before raising the bridge, but the boat can only pass when the bridge is raised. Deadlock. The solution: don't block the bridge -- let the boat through asynchronously.

Debugging Tips

Symptom	Cause	Fix
Application hangs indefinitely	Sync-over-async deadlock -- .Result or .Wait() inside async context	Replace .Result/.Wait() with await throughout the call chain
Exceptions from background work silently disappear	async void or unawaited Tasks discarding exceptions	Use async Task, await all tasks, or use safe fire-and-forget wrapper

Symptom	Cause	Fix
Parallel.ForEach returns before work completes	Async lambda treated as async void	Replace with Task.WhenAll or Parallel.ForEachAsync
Test hangs when calling async code	xUnit SynchronizationContext combined with .Result	Mark test methods as async Task and use await
ConfigureAwait(false) still deadlocks	Only one await has ConfigureAwait(false) -- others still capture context	Apply ConfigureAwait(false) to every await in the method chain

7.9. Task.Run vs async -- CPU-bound vs I/O-bound Work

Builds on: 7.1 (CPU-bound vs I/O-bound introduced there), 7.3 (async/await -- Task.Run integrates with it), 7.8 (pitfalls -- Parallel.ForEach with async covered there).

The core distinction

```
// I/O-bound -- just use async/await, no Task.Run needed:
public async Task<string> GetDataAsync()
{
    return await _httpClient.GetStringAsync(url);
    // Thread is released during HTTP call -- no CPU used while waiting
}

// CPU-bound -- use Task.Run to move work to thread pool:
public async Task<int[]> SortDataAsync(int[] data)
{
    return await Task.Run(() =>
    {
        Array.Sort(data);   // CPU actively working -- no I/O waiting
        return data;
    });
}
```

Task.Run -- freeing the calling thread

```
// UI button click -- must not block the UI thread:
private async void OnCalculateClick(object sender, EventArgs e)
{
    _button.Enabled = false;
    _label.Text = "Calculating...";

    // Move CPU work to thread pool -- UI thread freed immediately:
    var result = await Task.Run(() => PerformHeavyCalculation(_inputData));

    // Continuation runs back on UI thread -- safe to update UI:
    _label.Text = $"Result: {result}";
    _button.Enabled = true;
}

// In ASP.NET Core -- usually DON'T use Task.Run:
// Task.Run for CPU-bound work takes a thread pool thread away from
```

```
// handling requests, then your request waits for it.
// For pure CPU-bound work in ASP.NET Core, just call synchronously:
public IActionResult ProcessData([FromBody] DataDto dto)
{
    var result = _service.ProcessData(dto.Data);
    return Ok(result);
}
```

Mixing CPU and I/O bound work

```
// Sequential I/O then CPU:
public async Task<Report> GenerateReportAsync(int userId, CancellationToken ct)
{
    // I/O -- await directly:
    var orders = await _context.Orders
        .Where(o => o.UserId == userId)
        .ToListAsync(ct);

    // CPU -- run on thread pool if heavy enough:
    return await Task.Run(() => BuildReport(orders), ct);
}
```

SemaphoreSlim -- controlling concurrency

```
// Max 10 concurrent async operations:
private readonly SemaphoreSlim _semaphore = new SemaphoreSlim(10);

public async Task<Order[]> GetOrdersConcurrentlyAsync(
    int[] orderIds, CancellationToken ct)
{
    var tasks = orderIds.Select(async id =>
    {
        await _semaphore.WaitAsync(ct);   // acquire slot
        try
        {
            return await _repo.GetByIdAsync(id, ct);
        }
        finally
        {
            _semaphore.Release();          // release slot
        }
    });

    return await Task.WhenAll(tasks);
}
```

Parallel.ForEachAsync -- bounded parallel async work (.NET 6+)

```
// Process orders with max 5 concurrent operations:
await Parallel.ForEachAsync(
    orders,
    new ParallelOptions
    {
        MaxDegreeOfParallelism = 5,
        CancellationToken = ct
```

```
    },
    async (order, token) =>
    {
        await _processor.ProcessAsync(order, token);
    });

// Works with IAsyncEnumerable source too:
await Parallel.ForEachAsync(
    source: _context.Orders.AsAsyncEnumerable(),
    parallelOptions: new ParallelOptions { MaxDegreeOfParallelism = 10 },
    body: async (order, token) => await ProcessAndSaveAsync(order, token));
```

The misconception -- wrapping sync in Task.Run

```
// WRONG -- Task.Run does not make sync I/O truly async:
public Task<Order> GetByIdAsync(int id)
    => Task.Run(() => GetByIdSync(id));   // still blocks a thread during I/O

// CORRECT -- use genuinely async data access library:
public async Task<Order> GetByIdAsync(int id)
    => await _connection.QueryFirstOrDefaultAsync<Order>(
        "SELECT * FROM Orders WHERE Id = @id", new { id });
```

Key Takeaways for 7.9

- I/O-bound: use async/await with async APIs -- no Task.Run needed, no thread consumed during wait
- CPU-bound: use Task.Run to offload to thread pool -- frees the calling thread (especially on UI threads)
- In ASP.NET Core, Task.Run for CPU-bound work is rarely justified -- adds overhead without benefit
- Task.Run does not make synchronous I/O async -- use genuinely async libraries instead
- SemaphoreSlim controls concurrency in async code -- WaitAsync() is the async-safe acquire
- Parallel.ForEachAsync (.NET 6+) is the clean API for bounded parallel async work

```
Work type                 | In UI app              | In ASP.NET Core
--------------------------+------------------------+--------------------
I/O-bound (DB, HTTP, file) | await async API        | await async API
CPU-bound, short (<10ms)   | Call directly          | Call directly
CPU-bound, long (>50ms)    | await Task.Run(...)    | Usually directly
CPU-bound, many concurrent | Parallel.ForEachAsync  | Parallel.ForEachAsync
Sync blocking library      | await Task.Run(...)    | Avoid if possible
```

Memory aid: Task.Run is a courier service. You hand the package (CPU work) to a courier (thread pool thread) and immediately get a tracking number (Task). You don't wait at the door. But if the package needs to be fetched from another city (I/O work), you don't need a courier at all -- you just send an email and wait for the reply asynchronously.

Debugging Tips

Symptom	Cause	Fix
ASP.NET Core performance degrades under load	Task.Run used excessively -- stealing thread pool threads from request handling	Remove unnecessary Task.Run; let ASP.NET Core manage the thread pool
CPU work blocks UI thread despite Task.Run	Task.Run result awaited on UI thread before returning	Ensure all CPU work is inside Task.Run delegate; UI updates after await are fine
SemaphoreSlim deadlock	WaitAsync called but Release never called due to missing finally	Always release in a finally block
Parallel.ForEachAsync runs sequentially	MaxDegreeOfParallelism set to 1, or source is not async	Set MaxDegreeOfParallelism to desired concurrency level
Task.Run cancellation not stopping in-progress work	CancellationToken only prevents starting, not stops running work	Check ct.ThrowIfCancellationRequested() inside the Task.Run delegate periodically

---

Section 7 complete. All 9 roadmap topics (7.1 through 7.9) covered: Threads vs Tasks (7.1), Task and Task<T> (7.2), async/await (7.3), Task.WhenAll/WhenAny (7.4), CancellationToken (7.5), Exception handling in async (7.6), IAsyncEnumerable<T> (7.7), Common async pitfalls (7.8), Task.Run vs async (7.9). Say next to begin Section 8: Memory Management.